

How EE_GB_MGH_cubic_clean.mw Enters the Fortran Evolution Code

Code map for 4_dim_operators_flat_to_bh_GB_K_MGH

September 28, 2026

Purpose

The Maple worksheet `../EE_GB_MGH_cubic_clean.mw` derives the spherically symmetric 3+1 equations used by the Fortran code in the current directory. The worksheet defines the metric and foliation, derives the metric and gauge evolution equations, builds the generalized-harmonic/MGH and Gauss-Bonnet (GB) terms, derives the scalar equation, and finally exports Fortran expressions.

The Fortran implementation evaluates some direct evolution equations and then solves a 5 by 5 algebraic system generated from the worksheet. The generated system is contained in:

$$C(q, \partial_r q, \partial_r^2 q; r) X + S(q, \partial_r q, \partial_r^2 q; r) = 0,$$

where `A_matrix.f90` stores the coefficient matrix C , `Source.f90` stores the source/residual vector S , and the code solves for the five RHS components X .

Notebook Derivation Pipeline

The visible sections and printed messages in the worksheet show the following derivation chain.

Worksheet stage	Role in the code
Spacetime and foliation	Defines the spherical metric variables, unit normal, spatial metric, inverse spatial metric, extrinsic curvature, lapse and shift objects. These become the evolved state variables <code>grr</code> , <code>gT</code> , <code>Krr</code> , <code>KT</code> , <code>alp</code> , and <code>beta</code> .
Metric evolution equations	Produces first-order metric evolution equations. In the active code these appear directly as <code>RR(1)</code> and <code>RR(2)</code> in <code>eeqns.f90</code> .
Gauge evolution equations	Produces lapse and shift-driver equations. The implemented forms are <code>RR(5)</code> and <code>RR(6)</code> , including the code parameter <code>z1</code> .
Generalized harmonic constraints	Defines the constraint quantities. The worksheet note “the constraint is actually $K = \text{pip} \cdot (1 + \text{Atilde})$ ” corresponds to the trace-K diagnostic used in <code>rhs.f90</code> .
Energy-momentum tensor and projections	Defines scalar-field projections, MGH terms, and GB stress-energy contributions. These are used in the projected equations that feed the generated linear system.

Field equations	Builds <code>EEMGH</code> , <code>EEMGHnn</code> , <code>EEMGHgn</code> , and <code>EEMGHgg</code> projections. The worksheet notes that second metric derivatives on the right-hand side are replaced using GR plus matter expressions before export.
Scalar field evolution equation	Builds <code>GB</code> , the scalar-field derivative terms, and <code>ddphis</code> . In the active code, the first scalar equation is direct, while the second scalar/equation component enters the generated 5 by 5 solve through the fifth component.
Constraint equations	Builds Hamiltonian and momentum expressions, plus a constraint vector printed as <code>unknown(1:4)</code> in the worksheet. The current Fortran diagnostic implements only the last two simple constraints.
Export to Fortran	Provides the generated algebra. The parent directory contains fixed-form exports <code>../A_matrix.f</code> and <code>../Source.f</code> ; the current directory contains free-form include versions <code>A_matrix.f90</code> and <code>Source.f90</code> .

Runtime Flow

The equations enter the time evolution through this call chain:

`main.f90` $\rightarrow$ `evolve.f90` $\rightarrow$ `rhs.f90` $\rightarrow$ `eeqns.f90`.

`evolve.f90` implements a fourth-order Runge-Kutta update. Each RK stage computes finite-difference derivatives with `derivs.f90`, calls `rhs_time`, applies boundary conditions, adds dissipation, and updates the state. The key call is:

```
call rhs_time(uprev, du, ddu, urk, N, x, iHor)
```

Inside `rhs.f90`, `rhs_time` loops over grid points and calls:

```
call eeterms(u(:,i), du(:,i), ddu(:,i), RR, x(i), gam_factor(i, iHor))
```

Thus each point sees the local field values, first radial derivatives, second radial derivatives, the coordinate radius $r = x(i)$, and a coupling taper `gam_factor`. The taper is a code-level addition around the apparent horizon buffer: `rhs.f90` multiplies the Maple GB/MGH couplings through the `factor` argument before the generated equations are evaluated.

State and Parameter Map

The state vector has ten evolved variables. `params.f90` assigns the indices as:

Index	Code symbol	Meaning
1	<code>grr</code>	radial spatial metric component
2	<code>gT</code>	angular spatial metric factor
3	<code>Krr</code>	radial extrinsic curvature component
4	<code>KT</code>	angular extrinsic curvature component
5	<code>alp</code>	lapse
6	<code>beta</code>	radial shift
7	<code>pip</code>	scalar/trace variable used by the MGH system
8	<code>rho</code>	radial generalized-harmonic/scalar auxiliary variable
9	<code>sigma</code>	scalar derivative variable, called Sigma in the algebra
10	<code>phi</code>	scalar field

The main model parameters read from `pars.in` are declared in `params.f90`: `kappa`, `alpha`, `gam`, `Ahat`, `Atilde`, and `z1`. In `eeqns.f90`, the local generated-algebra variables `gam` and `alpha` represent the couplings γ and α , while `alp` is the lapse. Before evaluating the generated terms the code applies:

```
gam = gam0*factor
alpha = alpha0*factor
```

so the γ and α couplings can be smoothly turned off in the retained interior buffer behind the apparent horizon.

Direct Equations Implemented in `eeqns.f90`

The subroutine `eeterms` first copies the state vector and derivatives into the worksheet-style scalar names. This happens in `eeqns.f90`, lines 20–48.

Five RHS components are then assigned directly:

$$\begin{aligned}
\partial_t g_{rr} &= -2\alpha_{\text{lapse}} K_{rr} + \beta \partial_r g_{rr} + 2g_{rr} \partial_r \beta, \\
\partial_t g_T &= -2\alpha_{\text{lapse}} K_T + \beta \partial_r g_T + \frac{2}{r} \beta g_T, \\
\partial_t \alpha_{\text{lapse}} &= \alpha_{\text{lapse}}^2 z_1 \log\left(\frac{\sqrt{g_{rr}} g_T}{\alpha_{\text{lapse}}}\right) + \alpha_{\text{lapse}}^2 \text{pip} + \beta \partial_r \alpha_{\text{lapse}}, \\
\partial_t \beta &= -\alpha_{\text{lapse}} \beta z_1 \log\left(\frac{\sqrt{g_{rr}} g_T}{\alpha_{\text{lapse}}}\right) + \alpha_{\text{lapse}}^2 \rho - \alpha_{\text{lapse}} \beta z_1 + \beta \partial_r \beta - \frac{\alpha_{\text{lapse}}}{g_{rr}} \partial_r \alpha_{\text{lapse}}, \\
\partial_t \phi &= -\alpha_{\text{lapse}} \Sigma + \beta \partial_r \phi.
\end{aligned}$$

These are exactly the assignments at `eeqns.f90:50–55`:

```
RR(1) = -2*Krr*alp+beta*dxgrr+2*dxbeta*grr
RR(2) = -2*KT*alp+dxgT*beta+2/r*beta*gT
RR(5) = alp**2*log(sqrt(grr)*gT/alp)*z1+alp**2*pip+dxalp*beta
RR(6) = ...
RR(10) = -alp*Sigma+beta*dxphi
```

Generated 5 by 5 System

The remaining five RHS components are obtained from a pointwise 5 by 5 solve. The ordering is fixed by `eeqns.f90:63–67`:

$$X = \begin{bmatrix} \partial_t \text{pip} \\ \partial_t \rho \\ \partial_t K_{rr} \\ \partial_t K_T \\ \partial_t \Sigma \end{bmatrix}.$$

The wrapper `solve_rhs_system` is at `eeqns.f90:71–94`. It allocates a 5 by 5 matrix `cg` and a five-component source vector `unknown`, includes the generated blocks, changes the sign of the source, and solves:

```
cg(:, :) = 0.0d0
unknown(:, :) = 0.0d0

include 'A_matrix.f90'
include 'Source.f90'

rhs_system(:) = -unknown(:, 1)
call solve_linear_5x5(cg, rhs_system, info)
```

Therefore the generated algebra is interpreted as:

$$\text{cg} X = -\text{unknown}.$$

After `solve_linear_5x5` returns, `rhs_system` contains the solution for X , not the source.

Coefficient Matrix

`A_matrix.f90` fills the non-zero entries of `cg`. The non-zero pattern is:

row	non-zero columns
1	1, 3, 4, 5
2	2
3	1, 3, 4, 5
4	1, 3, 4, 5
5	3, 4, 5

These entries contain the metric variables, the lapse, the shift, the scalar field derivatives, and the parameters `Ahat`, `Atilde`, γ , and α . This is the compact generated representation of the coupled MGH/GB evolution equations after the worksheet has isolated the time derivatives.

Source Vector

`Source.f90` fills:

$$\text{unknown}(1, 1), \dots, \text{unknown}(5, 1).$$

The file uses Maple-generated temporaries `s1`, `s2`, ... to keep the expression sizes manageable in Fortran. The five assignments occur at `Source.f90`:47, 70, 121, 160, and 202. Although the variable name is `unknown`, in the active Fortran wrapper it is the residual/source vector on the right-hand side of the linear system.

Generated Files Versus Active Code

The parent directory contains fixed-form Fortran output:

- `../A_matrix.f`
- `../Source.f`

The current directory contains free-form include files:

- `A_matrix.f90`
- `Source.f90`

The local files keep the generated algebra but adapt it to the active module: the matrix name is `cg` instead of the exported `cg0`, and continuation syntax is changed from fixed-form `\#` continuations to free-form `\&` continuations. These files are included inside `solve_rhs_system` rather than compiled as separate modules.

Gravity Sector Field Equations

The worksheet defines the gravitational field equations through the Maple object `EEMGH`. In tensor notation the definition is

$$R_{ab} + \frac{1}{2}g_{ab}(M^c{}_c + T^c{}_c) = M_{ab} + T_{ab},$$

where M_{ab} denotes `MGH_term\{a b\}` and T_{ab} is the total energy-momentum tensor. The trace terms $M^c{}_c$ and $T^c{}_c$ are the corresponding mixed-index traces. Equivalently, the residual form used for projections is

$$E_{ab}^{\text{MGH}} \equiv R_{ab} + \frac{1}{2}g_{ab}(M^c{}_c + T^c{}_c) - M_{ab} - T_{ab} = 0.$$

The worksheet then projects this equation along the unit normal and spatial metric to build `EEMGHnn`, `EEMGHgn`, and `EEMGHgg`. Those projected equations are the gravitational-sector inputs to the generated 5 by 5 system. The T_{ab} contribution contains the scalar K-essence terms proportional to γ , while the Gauss-Bonnet contributions are the terms proportional to α .

Scalar Energy-Momentum Terms

The scalar-field stress tensor used by the worksheet is not purely canonical. It is built from the kinetic scalar

$$X = \nabla_c \phi \nabla^c \phi,$$

which in the spherical 3+1 variables is represented locally as

$$X = -\Sigma^2 + \frac{(\partial_r \phi)^2}{g_{rr}}.$$

The worksheet defines the K-essence function

$$K(X) = -\frac{1}{2}X - \frac{\gamma}{8}X^3,$$

so the terms proportional to γ are the nonlinear K-essence terms. In mixed-index form the scalar contribution can be read as

$$T^a_b = \nabla^a \phi \nabla_b \phi - \frac{1}{2} \delta^a_b X + \frac{3\gamma}{4} X^2 \nabla^a \phi \nabla_b \phi - \frac{\gamma}{8} \delta^a_b X^3.$$

The last two terms are the K-essence nonlinearities. Separately, terms proportional to α are the Gauss-Bonnet terms; these enter through the GB stress-energy contribution and through the generated equations in `A_matrix.f90` and `Source.f90`.

Scalar Field Equations

The worksheet defines the Gauss-Bonnet scalar as

$$\mathcal{G} = R_{abcd}R^{abcd} - 4R_{ab}R^{ab} + R^2.$$

With the K-essence function above,

$$K_X \equiv \frac{\partial K}{\partial X} = -\frac{1}{2} - \frac{3\gamma}{8}X^2,$$

the covariant scalar-field equation used in the worksheet is

$$\mathcal{E}_\phi \equiv \nabla_a (K_X \nabla^a \phi) - \frac{\alpha}{2} \mathcal{G} = 0.$$

Thus γ changes the scalar kinetic current through K_X , while α multiplies the Gauss-Bonnet source.

In the first-order variables used by the evolution code, the first scalar equation is explicit:

$$\partial_t \phi = \beta \partial_r \phi - \alpha_{\text{lapse}} \Sigma.$$

This is implemented directly as `RR(10)` in `eeqns.f90`. The second scalar equation determines $\partial_t \Sigma$. In the generated algebra it is the fifth row of the 5 by 5 system,

$$C_{53} \partial_t K_{rr} + C_{54} \partial_t K_T + C_{55} \partial_t \Sigma + S_5 = 0,$$

and the code stores the solved value as `RR(isigma)`. The coefficient of $\partial_t \Sigma$ shows explicitly where the K-essence parameter enters the principal scalar equation:

$$C_{55} = \frac{(15\gamma\Sigma^4 + 4) g_{rr}^2 - 18\gamma\Sigma^2 (\partial_r \phi)^2 g_{rr} + 3\gamma (\partial_r \phi)^4}{8 \alpha_{\text{lapse}} g_{rr}^2}.$$

Constraints and Diagnostics

The worksheet's final Fortran-print section shows a constraint vector:

```
unknown(1) = Hamiltonian-like expression
unknown(2) = momentum-like expression
unknown(3) = -rho + dxgrr/(2*grr**2) - dxgT/(gT*grr)
            - 2/(r*grr) + 2/(gT*r)
unknown(4) = pip + 2*KT/gT + Krr/grr
```

The current diagnostic routine `constraints` implements only the last two of these. See `rhs.f90`, lines 101–104:

```
! Cne(1:2,:) are reserved for Hamiltonian and momentum constraints.
Cne(3,i) = -rho + dxgrr/(2.0d0*grr**2) - dxgT/(gT*grr) &
          & - 2.0d0/(xm*grr) + 2.0d0/(gT*xm)
Cne(4,i) = pip + 2.0d0*KT/gT + Krr/grr
```

This means `CGH` and `CTGH` in the output correspond to the generalized-harmonic radial constraint and the trace-K constraint. The `Hamiltonian` and `Momentum` SDF outputs are reserved in `main.f90`, but in the current code `Cnew(1,:)` and `Cnew(2,:)` remain zero unless those expressions are added.

Initial Data

`initial.f90` builds conformally flat Schwarzschild-like initial data:

$$\psi(r) = 1 + \frac{M}{2r}, \quad g_{rr} = g_T = \psi^4, \quad K_{rr} = 0, \quad K_T = 0.$$

It also sets:

$$\text{pip} = 0, \quad \rho = -\frac{2}{\psi^5} \partial_r \psi.$$

The comment at `initial.f90:55` states that these choices satisfy the initial trace-K and ρ constraints, i.e. the two constraints actually implemented in `rhs.f90`.

Important Implementation Notes

- The worksheet-derived active evolution equations enter primarily through `A_matrix.f90` and `Source.f90`, included at `eeqns.f90:88–89`.
- The direct metric, gauge, and scalar first-order equations are hard-coded in `eeqns.f90:50–55`.
- The code solves the generated system pointwise at every RK substage and every active grid point.
- The γ and α couplings, stored in code as `gam` and `alpha`, are multiplied by `gam_factor` in `eeqns.f90:22–23`, so the numerical code applies an apparent-horizon coupling taper that is separate from the Maple derivation.
- The names are easy to confuse: `alp` is the lapse, while `alpha` is the α coupling parameter used by the generated algebra.
- The temporary variables `PiKrr`, `PiKT`, `Pipip`, and `Pirho` remain declared in `tempvars.f90`, but the active code does not use them. The actual RHS solution is stored in `rhs_system`.

Practical Regeneration Checklist

If the worksheet is changed and the generated equations are exported again, the code path to update is:

1. Export the coefficient matrix and source vector from the worksheet.
2. Convert the fixed-form output to the free-form include style used by `A_matrix.f90` and `Source.f90`.
3. Ensure the matrix variable is named `cg`. The source/residual vector should have five rows and one column, matching the current `unknown(5,1)` shape.

4. Preserve the ordering

$$(\partial_t \text{pip}, \partial_t \rho, \partial_t K_{rr}, \partial_t K_T, \partial_t \Sigma).$$

5. Rebuild and test that `solve_linear_5x5` does not report a zero or near-zero pivot.
6. If full Hamiltonian and momentum diagnostics are desired, add the worksheet's `unknown(1)` and `unknown(2)` constraint exports to `rhs.f90`; they are not currently active.